SEQUENTIAL LINEAR PROGRAMMING FOR NONLINEAR POOLING OPTIMIZATION PROBLEMS

A Literature Review and Re-Implementation of Sequential Linear Programming for the Pooling Problem

Group ID	9
Group Members	Razzan Tambe (15) · razzan.tambe@somaiya.edu
	Saurabh Tripathi (04) · saurabh.at@somaiya.edu
	Vivek Sharma (12) · vivek.sharma2@somaiya.edu
Main Implementation	Sequential Linear Programming with LP trust-region subproblems

Selected Research Paper

A. Grothey and K. McKinnon, "On the effectiveness of sequential linear programming for the pooling problem," *Annals of Operations Research*, vol. 322, pp. 691–711, 2023.

Abstract

Optimization is concerned with selecting decision variables so that a measurable objective is minimized or maximized while operational, physical, logical, and quality requirements are satisfied. Nonlinear optimization becomes substantially more difficult when constraints contain products of decision variables, because the feasible region can be non-convex and several locally optimal solutions may exist. This report studies this class of problems through a focused literature review and a detailed implementation of Sequential Linear Programming (SLP) for the pooling problem.

The pooling problem represents the mixing of raw materials through intermediate pools or bins to produce final products whose nutrient or quality compositions must remain within specified limits. The intermediate pool composition is itself a decision variable, so terms such as a pool proportion multiplied by a flow quantity create bilinear expressions. The selected study by Grothey and McKinnon (2023) investigates randomized local optimization methods for fixed-demand pooling instances and compares SLP with Sequential Quadratic Programming, an interior-point method, and the global solver BARON. The paper emphasizes solution quality within practical time budgets and the effect of multiple starting points in a non-convex landscape.

An independent Python implementation was developed using NumPy, SciPy, pandas, and Matplotlib, with the open-source HiGHS solver in place of CPLEX. It is a filter-SLP method: at each iteration the bilinear qq-formulation is linearized around the current point, an LP trust-region subproblem is solved, and a filter on objective and constraint violation decides whether the trial point is accepted. Twenty randomized starts were run on the real animal-feed instance af-1 of the paper (16 raw materials, 6 bins, 25 products, 8 nutrients). All 20 runs were feasible and within 0.2% of the best known value. The best run obtained 296.44604, identical to the authors' value, with a constraint violation of about 5×10^{-10}; the other runs stopped at nearby local solutions up to 0.15% higher.

The study is limited to af-1: the complete af-series experiment (500 starts per instance) is computationally expensive and was not repeated, so only af-1 was run, with 20 starts, and no claim is made about the other instances. Times, iteration counts and %Good are not expected to match the paper exactly, because HiGHS replaces CPLEX and the starting-point generator is our own choice. The report separates published findings from the group's own experiments and makes no claim of a new algorithmic contribution.

Keywords: nonlinear optimization; pooling problem; sequential linear programming; trust region; non-convex optimization; randomized multistart; constrained optimization

1. Introduction

1.1 Problem Context and Motivation

Optimization provides a mathematical framework for making decisions when multiple feasible choices exist but resources, quality requirements, capacities, or physical relationships restrict those choices. In a simple linear program, the objective and constraints are linear and efficient algorithms can often obtain globally optimal solutions. Many practical systems, however, contain nonlinear relationships. When a variable affects both a flow and a composition, when demand depends nonlinearly on price, or when a physical model contains nonlinear equations, the resulting problem may become a nonlinear program (NLP) or a mixed-integer nonlinear program (MINLP).

The pooling problem is a representative non-convex optimization problem. Raw materials with different nutrient compositions are blended through intermediate pools and then allocated to final products. A pool has a composition determined by its mixture proportions, while the quantity of that pool sent to each product is another decision. Their product creates a bilinear relationship. Quality requirements are therefore nonlinear even when demand is fixed and costs are otherwise simple.

Sequential Linear Programming addresses such problems by solving a sequence of local linear approximations. Instead of asking a linear program to represent the complete nonlinear model, SLP uses the current point to construct a first-order approximation, restricts the step with a trust region, solves the resulting LP, and then evaluates the original nonlinear functions at the trial point. This repeated local modeling makes the method comparatively transparent and computationally practical, although it remains a local optimization approach.

The selected pooling study is particularly suitable for an IA-1 re-implementation because it combines a clear mathematical model with an algorithm that can be demonstrated step-by-step. It also highlights an important practical issue: different randomized starting points can reach different local solutions. Consequently, the quality of an optimization method cannot be judged only by whether one run terminates successfully; feasibility, objective quality, runtime, robustness, and sensitivity to initialization must also be considered.

1.2 Objectives of the Study

- To review recent research on nonlinear and constrained optimization methods relevant to the pooling problem and identify the major algorithmic approaches used.
- To compare the reviewed studies in terms of optimization problem, mathematical structure, algorithm, computational setting, evaluation criteria, and limitations.
- To formulate the fixed-demand pooling problem mathematically, including decision variables, objective function, demand constraints, pool convexity, and nonlinear product-quality constraints.
- To understand and implement the core Sequential Linear Programming procedure using local linearization and an infinity-norm trust region.
- To validate the implementation through convergence, feasibility, randomized multistart results, and product-quality visualizations.
- To compare the behavior of the implementation with the computational findings reported by Grothey and McKinnon (2023) on the af-1 instance, separating the CPLEX and HiGHS environments.

2. Literature Review

Recent research in nonlinear and constrained optimization explores different approaches for solving complex optimization problems, including Sequential Linear Programming (SLP), metaheuristics, branch-and-bound, interior-point methods, first-order methods, and active-set techniques. The six selected studies address different optimization problems and evaluate their methods using computational performance, solution quality, convergence, and problem-specific measures. The studies also consider different datasets or benchmark instances and discuss limitations such as computational cost, parameter dependence, local optimality, and problem-specific assumptions.

The literature comparison is presented in Table 1, covering the problem addressed, optimization technique, dataset or problem instances, evaluation measures, major findings, and limitations. Based on this review, Grothey and McKinnon (2023) was selected for detailed study because it directly addresses the pooling problem using Sequential Linear Programming, which is the focus of this report.

2.1 Literature Review of Selected Papers

Study	Optimization problem	Main method	Data / instances	Key findings / evaluation	Main limitation
Terzi et al. (2024)	Non-convex MINLP for joint lot-sizing and dynamic pricing	GA, Late Acceptance Hill Climbing, Simulated Annealing, matheuristics	Real-world-inspired production/pricing instances	Profit and computational performance; structured matheuristics perform strongly	Heuristic sensitivity to encoding, parameters, and search budget
Grothey & McKinnon (2023)	Non-convex pooling NLP with bilinear mixing relationships	Randomized SLP, SQP, interior-point methods; BARON for global comparison	Industrial animal-feed pooling instances	Feasibility, solution quality, and time-to-quality under randomized starts	SLP is local; results depend on starting point
Sass et al. (2024)	Large-scale NLP/MINLP parameter estimation	Spatial branch-and-bound with progressively growing datasets	Chemical engineering and biochemical models	Global optimization with reduced CPU burden from staged data growth	Savings depend on how representative the reduced dataset is
Gill & Zhang (2024)	Nonlinearly constrained optimization / bound-constrained subproblems	Shifted primal-dual projected-search interior-point method	CUTEst and other constrained test problems	Robustness and reduced search-direction computations	Higher implementation and linear-algebra complexity
Muehlebach & Jordan (2026)	Constrained nonlinear optimization, including non-convex settings	Accelerated first-order methods with local feasible-set approximations	Compressed sensing and sparse regression	Stationarity and convergence-rate analysis	Practical behavior depends on smoothness, geometry, and regularity assumptions
di Serafino et al. (2024)	Nonlinear optimization with polyhedral constraints	Stationarity decomposition and active-set algorithm	Convex quadratic-program numerical setting	More informative stationarity measures and active-manifold behavior	More specialized constraint geometry and problem class

2.2 Comparative Critical Analysis

The six studies address different nonlinear or constrained optimization problems, but they share the objective of obtaining good-quality solutions while controlling computational effort. Their main differences are in the mathematical formulation, optimization strategy, problem size, and evaluation criteria.

The studies by **Terzi et al.** and **Grothey and McKinnon** deal with non-convex optimization problems where local solution quality and computational time are important. Terzi et al. use metaheuristic and matheuristic approaches, whereas Grothey and McKinnon investigate local mathematical optimization methods such as SLP, SQP, and interior-point methods. In contrast, **Sass et al.** use spatial branch-and-bound with growing datasets, giving greater emphasis to global optimization and computational efficiency.

Gill and Zhang use a primal-dual interior-point framework with projected search, while **Muehlebach and Jordan** focus on accelerated first-order methods for nonlinear constraints. **di**

Serafino et al. take a more specialized approach by studying stationarity and active constraints in problems with polyhedral constraints. Thus, the methods range from heuristic search to mathematically structured local and global optimization techniques.

The studies also differ in their problem instances and evaluation measures. The reviewed works use industrial, real-world-inspired, chemical/biochemical, and mathematical benchmark problems. Depending on the study, performance is evaluated using **objective value or solution quality, feasibility, convergence, computational time, CPU effort, or stationarity measures**. This makes direct numerical comparison between the papers inappropriate because their formulations, datasets, and evaluation criteria are different.

A common limitation across the literature is that each method relies on assumptions or conditions related to the problem structure. Metaheuristics can depend on parameter settings and computational budgets, local methods may not provide global-optimality guarantees, and more structured global or constrained methods can involve greater computational or implementation complexity. Overall, the literature shows that optimization method selection is strongly influenced by the formulation, constraints, problem size, and required type of solution.

3. Optimization Problem Formulation

The pooling model considered in the implementation follows the fixed-demand structure described by Grothey and McKinnon. Let I be the set of raw materials, M the set of mixing pools, P the set of final products, and N the set of nutrients or quality attributes. The raw-material composition r_{in} is known. Product p has fixed demand t_p and nutrient bounds $\underline{d}_{pn}$ and $\bar{d}_{pn}$.

3.1 Decision Variables

The proportional variable λ_{mi} represents the fraction of pool m originating from raw material i. The flow f_{mp} is the amount sent from pool m to product p. The direct flow f^S_{ip} is the amount of raw material i sent directly to product p. The Python implementation stores the equivalent qq-formulation variables (bin and product fractions) in one vector x.

Pool nutrient composition is a weighted combination of raw-material compositions:

$$v_{mn} = \Sigma_{i \in I} \lambda_{mi} r_{in}, \qquad \forall m,n.$$

Because λ is proportional, each pool satisfies the convexity condition:

$$\Sigma_{i \in I} \lambda_{mi} = 1, \qquad \lambda_{mi} \geq 0, \qquad \forall m.$$

3.2 Objective Function

For fixed product demands, the implementation minimizes the cost of supplying products through pools and direct raw-material flows. The pooled contribution contains the bilinear term $\lambda_{mi} f_{mp}$:

$$\min_{f \geq 0, \lambda \geq 0} \Sigma_{i \in I} \left[c_i \Sigma_{m \in M, p \in P} \lambda_{mi} f_{mp} + c^S_i \Sigma_{p \in P} f^S_{ip} \right].$$

A lower objective value represents lower modeled production cost for the af-1 instance, provided all demand and quality constraints are satisfied.

3.3 Constraints

- **Product demand:** $\Sigma_{i \in I} f^S_{ip} + \Sigma_{m \in M} f_{mp} = t_p$, for every product p.
- **Product quality:** $\underline{d}_{pn} \leq [\Sigma_{i,in} r_{in} f^S_{ip} + \Sigma_{m,i,in} r_{in} \lambda_{mi} f_{mp}] / t_p \leq \bar{d}_{pn}$, for every product p and nutrient n.
- **Pool convexity:** the proportions in each pool must sum to one and remain non-negative.
- **Flow non-negativity:** all pooled and direct flows are non-negative.

3.4 Mathematical Structure and Non-Convexity

The product-quality expression contains the product $\lambda_{mi} f_{mp}$. A product of two decision variables is bilinear, so the original optimization problem is nonlinear and generally non-convex. This distinction is fundamental: SLP does not solve the original nonlinear problem in one step. It repeatedly solves local linear approximations and evaluates the original nonlinear functions at trial points. Different starting points can therefore lead to different local solutions.

4. Selected Research Paper

Grothey, A., and K. McKinnon (2023). "On the effectiveness of sequential linear programming for the pooling problem," *Annals of Operations Research*, vol. 322, pp. 691–711. DOI: 10.1007/s10479-022-05156-7.

4.1 Reason for Selection

- The pooling problem has clearly defined decision variables, objective terms, demand constraints, pool-composition constraints, and nonlinear product-quality constraints.
- The SLP mechanism can be implemented transparently with a standard LP solver, making the method suitable for step-by-step demonstration.
- The selected study compares multiple local methods and a global method, providing useful context for understanding the role of local optimization and time-to-quality.
- The study explicitly considers randomized starting points, which can be reproduced on the published af-series instances to expose multiple local solution basins.
- The af-series instances are real animal-feed data that are available for download, and the method needs only an open-source LP solver (HiGHS here, CPLEX in the paper).

4.2 Problem and Methodology Understanding

The selected paper considers the pooling problem, in which raw materials are mixed through intermediate pools to produce final products while satisfying quality or nutrient limits. Since the composition of an intermediate pool depends on the decision variables, the resulting mixing constraints contain bilinear terms, making the problem non-convex and capable of having multiple local solutions.

The main method studied is Sequential Linear Programming (SLP). At each iteration, the nonlinear problem is locally approximated by a linear programming subproblem. A step is obtained from this linearized problem, the solution is updated, and the process is repeated until convergence. Because SLP is a local method, the paper uses randomized starting points to explore different regions of the solution space.

The paper compares SLP with Sequential Quadratic Programming (SQP), Interior Point methods, and the global solver BARON. The experiments use industrial animal-feed pooling instances and compare methods mainly according to the quality of the solution obtained within a specified computational time. The study also examines different pooling formulations, including the proposed qq-formulation for fixed-demand problems.

The paper shows that SLP can obtain high-quality solutions efficiently within a limited time budget, although it does not provide a general guarantee of global optimality. This makes the study particularly useful for understanding the practical trade-off between solution quality, computational effort, and global optimality in non-convex optimization.

5. Methodology / Algorithm

Sequential Linear Programming solves a nonlinear constrained optimization problem by repeatedly replacing the nonlinear model with a linear local approximation. Consider:

$$\min_x f(x) = c^T x \text{ subject to } g(x) = 0,\ l \le x \le u$$

Here $x = (\lambda, \mu, V, D)$ collects the bin fractions, product fractions, mixing variables and product qualities of the qq-formulation, $f(x) = c^T x$ is the total cost (tons × price), and $g(x) = 0$ are the bilinear mixing equations $D = \Sigma\, \mu R + \Sigma\, \mu V$.

At iteration k, the bilinear products $\mu \cdot V$ are linearized around $x^{(k)}$ (the objective is linear and is not approximated):

$$g(x^{(k)} + \Delta x) \approx g(x^{(k)}) + J(x^{(k)})\Delta x,\ \partial(\mu V)/\partial\mu = V,\ \partial(\mu V)/\partial V = \mu.$$

The linear subproblem is: $\min_{\Delta x} c^T \Delta x$

$$\text{subject to } g(x^{(k)}) + J(x^{(k)})\Delta x = 0,\ l \le x^{(k)} + \Delta x \le u,\ |\Delta x_j| \le \rho^{(k)} w_j.$$

The infinity-norm trust region is the component-wise bound $-\rho^{(k)} w_j \le \Delta x_j \le \rho^{(k)} w_j$, where $w_j = u_j - l_j$ is the range of variable j ($\rho_0 = 0.5$, $\rho \le \rho_{max} = 1$); the bounds $l \le x \le u$, including non-negativity, are kept inside the LP.

5.1 Algorithm Steps

1. Initialize. Choose a random starting point $x^{(0)}$ (random convex combinations of the raw materials, derived variables consistent), set $\rho = \rho_0 = 0.5$, an empty filter and the stopping tolerances.

2. Evaluate. Compute the original nonlinear objective $f(x) = c^T x$ and the constraint violation $h(x)$, the scaled ℓ_1 norm of the mixing-equation residuals, at $x^{(k)}$.

3. Linearize. Compute the Jacobian of the bilinear terms $\mu \cdot V$ analytically at the current point ($\partial/\partial\mu = V$, $\partial/\partial V = \mu$); the objective is linear.

4. Solve LP. Solve the linearized trust-region LP for Δx with HiGHS (basis warm-started). If it is infeasible, run the restoration: alternating exact LPs (μ fixed, then the bins), and an elastic ℓ_1 LP if these make no progress.

5. Trial point. Set $x_{trial} = x^{(k)} + \Delta x$ and evaluate the original nonlinear $f(x_{trial})$ and $h(x_{trial})$, and the predicted reduction $\Delta q = f(x^{(k)}) - c^T x_{trial}$.

6. Stop test. If the LP was feasible, $h(x^{(k)}) \le 10^{-6}$ and the step or Δq is $\le 10^{-9}$ (Δq relative to $1 + |f|$), stop: $x^{(k)}$ is the solution of this start.

7. Acceptance. Accept x_{trial} if it gives sufficient decrease and is not dominated by the filter: for a step that reduces f as predicted, f falls by at least $\sigma = 0.1$ of Δq; otherwise $h \le \beta \cdot h(x^{(k)})$ or $f \le f(x^{(k)}) - \gamma \cdot h(x_{trial})$ ($\beta = 0.99$, $\gamma = 0.01$). Otherwise reject it.

8. Trust-region update and repeat. After an accepted step that reached the trust-region boundary, $\rho \leftarrow \min(\rho_{max}, 2\rho)$; after a rejected step, x is kept and $\rho \leftarrow 0.5 \cdot \min(\rho, \|\Delta x\|)$. Continue with step 2 until the stop test holds, $\rho < 10^{-9}$ or 400 iterations.

5.2 Methodology Flowchart

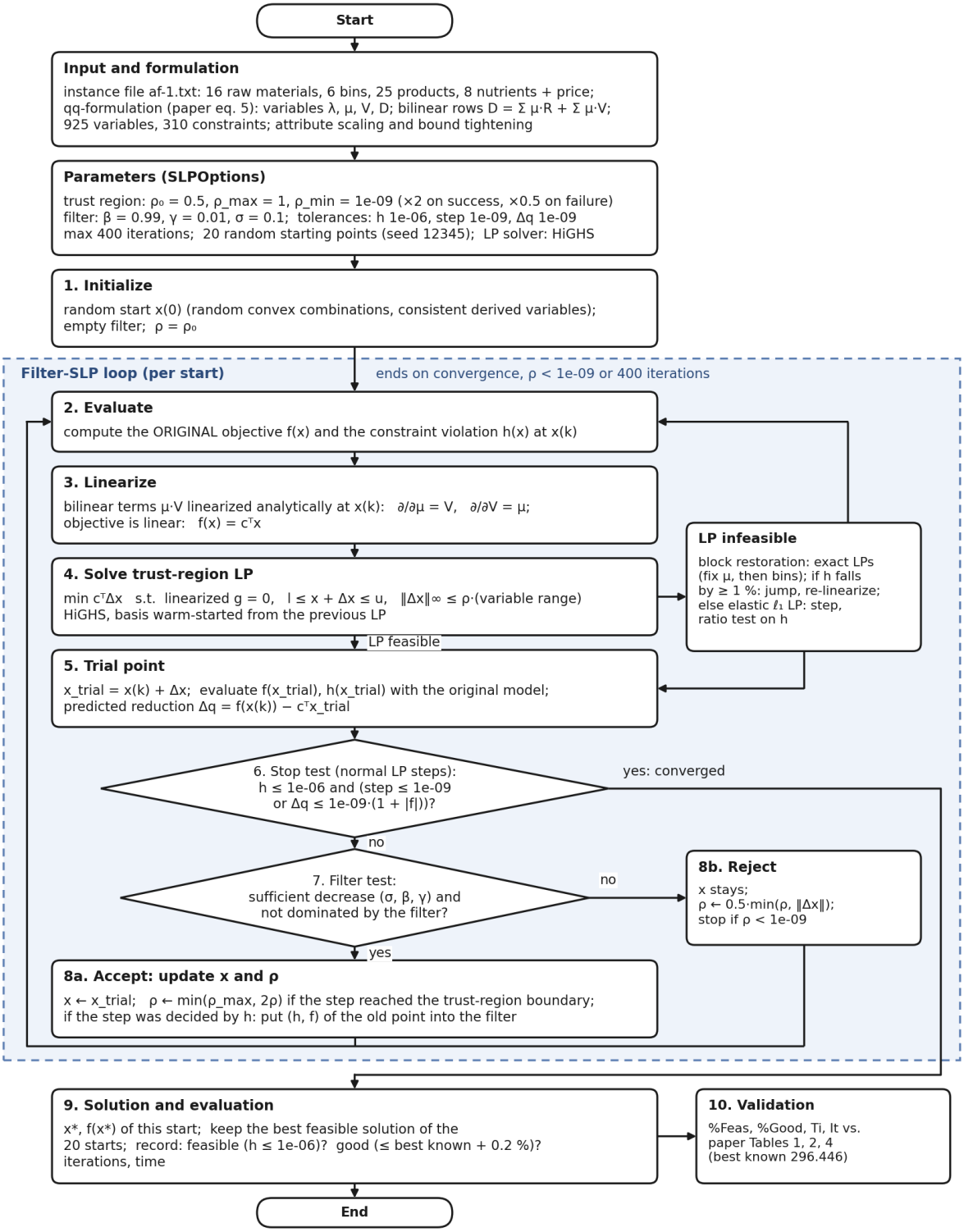


Figure 1. Methodology flowchart for the implemented Sequential Linear Programming procedure.

6. Implementation

The implementation was written in Python and designed to expose the core optimization process rather than hide it inside a black-box nonlinear solver. NumPy stores the model arrays, SciPy provides sparse matrices and the HiGHS linear-programming solver (used through its basis warm-start interface), pandas records the multistart results, and Matplotlib produces the validation figures.

6.1 Implementation Environment

Item	Details
Programming Language	Python 3.x
Libraries/Tools	NumPy, SciPy (sparse matrices, HiGHS LP solver), pandas, Matplotlib
Hardware / Runtime	Jupyter notebook on a standard CPU (HiGHS run single-threaded)
Dataset / Problem Instance	Real animal-feed instance af-1 of the paper: 16 raw materials, 6 bins, 25 products, 8 nutrients; 925 variables and 310 constraints in the qq-formulation
Algorithm Parameter	Randomized multistart, 20 starting points (seed 12345); filter-SLP, infinity-norm trust region (ρ doubled after full steps, halved after rejections); filter parameters $\beta = 0.99$, $\gamma = 0.01$, $\sigma = 0.1$
Stopping condition	Constraint violation $\le 10^{-6}$ (scaled) and step or predicted reduction $\le 10^{-9}$; at most 400 iterations

6.2 Implementation Evidence

The af-1 data file gives the nutrient matrix R (row = raw material, column = nutrient); row RM01 reads:

$$R_{RM01} = [10.5, 1.79, 5.5, 0.06, 0.3, 76.43, 0.15, 0.12].$$

Each raw material has a pooled-use cost c (0.17 to 2.985 per unit) and a direct-use (straight) cost c^S that is twice as high. The 25 products have a total demand of 400 t (20 products with positive demand) and lower and upper bounds for each of the 8 nutrients; for example DE06 (50 t) needs NU01 between 14.58 and 17.82 and NU06 between 63.9 and 78.1. The data are used exactly as provided in the file.

```
inst = load_instance("data/af-1.txt")
mod  = QQModel(inst)                                        # qq-formulation: 925 variables, 310 constraints
# I = 16 raw materials, M = 6 bins, P = 25 products, N = 8 nutrients (+ price column)

def objective(self, x):                                     # QQModel: total cost = sum_p tons_p * price_p
   return float(self.attr_scale[self.N] * (self.tons @ x[self.D_var[:, self.N]]))

def linearization(self, x0):                                # first-order model of the bilinear terms mu*V at x0
   mu0, V0 = self._mu_V(x0)
   coo_vals = np.concatenate([self.static_vals, -V0, -mu0])     # d/dmu = -V0, d/dV = -mu0
   rhs = self.rhs_lin.copy()
   np.subtract.at(rhs, self.bl_row, mu0 * V0)
   data = coo_vals[self._perm]
   return data, rhs
```

Figure 2: code evidence (excerpt of qq_model.py)

These methods encode the objective and the linearized bilinear mixing constraints that are re-solved as an LP at every SLP iteration. The complete source files contain the remaining constraint functions, the filter-SLP loop, the randomized multistart experiment, CSV export, and plotting code.

```
def solve(self, x_start) -> SLPResult:                                  # FilterSLP.solve, abridged
  filt = []                                                             # empty filter
  rho = o.rho0                                                          # initial trust region
  for k in range(1, o.max_iter + 1):
    lo = np.maximum(M.lb, x - rho * M.width)                            # infinity-norm trust region
    hi = np.minimum(M.ub, x + rho * M.width)
    data, rhs = M.linearization(x)                                      # LP coefficients at the current point
    st, xt = self.lp.solve(c, lo, hi, rhs, data)                        # HiGHS LP with warm-started basis
    if st == "infeasible":                                              # feasibility restoration
      xr, hr = self._block_restoration(x)                               # alternating exact LPs, then elastic LP
    ft = M.objective(xt)                                                # evaluate the ORIGINAL model at xt
    dq = f - float(c @ xt)                                              # predicted reduction of the LP
    accept = (f - ft >= o.sigma * dq) and self._acceptable(filt, ht, ft)  # filter test
    x, f, h, hinf = xt, ft, ht, htinf                                   # accepted: move to the trial point
    rho = min(rho, step) * o.shrink if step > 0 else rho * o.shrink     # rejected: shrink the trust region
```

Figure 3: Main algorithm evidence (abridged from slp.py)

7. Results and Validation

7.1 Experimental Results

Experiment / Instance	Parameter Setting	Objective Value	Metric(s)	Observation
Single SLP convergence run	Best of the 20 random starts (run 11); filter-SLP with infinity-norm trust region	296.44604	Constraint violation ≈ $\mathbf{4.95 \times 10^{-10}}$; 49 iterations (35 accepted, 13 rejected)	Converged from 810.9 to a feasible solution at the best known value.
20-run multistart experiment	20 randomized starting points; same SLP parameters	Best: 296.44604	20/20 feasible runs; 20/20 within 0.2% of the best known value	All runs produced feasible solutions; four runs reached 296.44604 exactly, the others stopped at nearby local solutions (up to 0.15% higher).
Most frequent solution	Randomized multistart	296.4514	Feasibility satisfied (11 of 20 runs)	Most starts ended at 296.4514 (0.002% above the best), showing the local nature of SLP.
Best obtained solution	Multistart SLP	296.44604	Constraint violation ≈ $\mathbf{4.95 \times 10^{-10}}$	The best run (run 11) matched the best known objective of the paper and satisfied all pooling quality constraints.

The multistart results show that 16 of the 20 runs ended within 0.01% of the best known value (11 at 296.4514, four at 296.44604, one at 296.4554) and the other four between 296.63 and 296.89 (at most 0.15% higher). Because every run was feasible, the differences are not caused by infeasibility; they reflect the non-convex local-solution behavior emphasized in the selected study: randomized starts can reach different local solutions.

7.2 Comparison with Reported Results

Measure	Reported in Paper	Our Implementation	Difference / Remark
Problem instances	Industrial animal-feed pooling instances	Same af-1 instance	Same data as the paper; instance sizes match Table 1 (925 variables, 310 constraints).
Optimization method	SLP with randomized starting points, compared with SQP, Interior Point and BARON	Filter-SLP with randomized multistart (HiGHS as LP solver)	Core filter-SLP method reproduced; HiGHS replaces CPLEX (72.5 vs 38.0 iterations per run on af-1).
Solution quality	Best known 296.44604; SLP-CPLEX: 79.0% of runs good (within 0.2%)	Best objective = 296.44604; 100% of runs good	Objective reproduced exactly on af-1; %Good depends on the start generator.
Acceptance logic	Filter strategy	Filter strategy	Same acceptance logic (filter with restoration).
Number of runs	500 randomized starting points per instance (Tables 2 and 4)	20 randomized starting points on af-1	Smaller experimental scale (20 vs 500 runs, af-1 only).
Feasibility	Feasible runs on af-1: 100%	**20/20 runs feasible**; best violation ≈ $\mathbf{4.95 \times 10^{-10}}$	Same feasibility rate as the paper on af-1.

The implementation reproduces the core computational mechanism of SLP and, on the same data, the best known objective of the paper on af-1 (296.44604), so objective values are directly comparable. Scope: the complete af-series experiment (500 starts per instance) is computationally expensive and was not repeated, so only af-1 was run, with 20 starts; no claim is made about the other instances. Run times, iteration counts and %Good are not expected to match exactly because HiGHS replaces CPLEX and the starting-point generator is our own choice; our runs need about twice as many iterations (72.5 versus 38.0 on af-1). The observed multistart behavior is consistent with the paper's emphasis on randomized starting points and the local nature of SLP.

7.3 Graphs / Visualizations

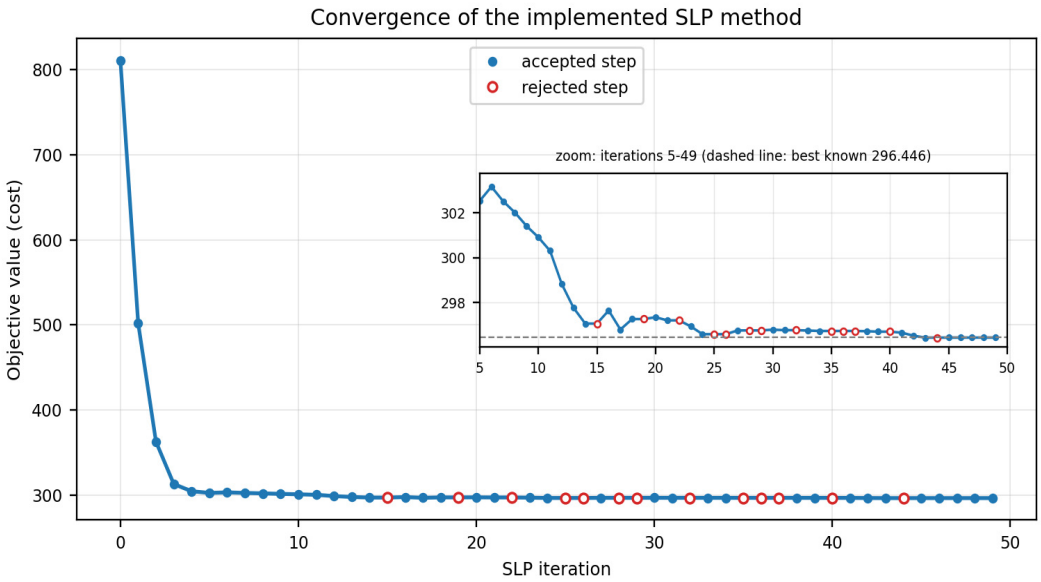


Figure 4. Objective value during one SLP run on af-1 (best of the 20 runs): from 810.9 at the random start to 296.44604 after 49 iterations. Open red circles mark rejected steps; the inset zooms in on iterations 5–49.

The convergence curve demonstrates the role of sequential local models. The objective falls from 810.9 at the starting point to about 302.5 after five iterations and then creeps down to 296.446, and the constraint violation falls to numerical zero. Thirteen of the 48 trial steps were rejected: when a trial step does not improve the original nonlinear model enough, the algorithm does not blindly move forward.

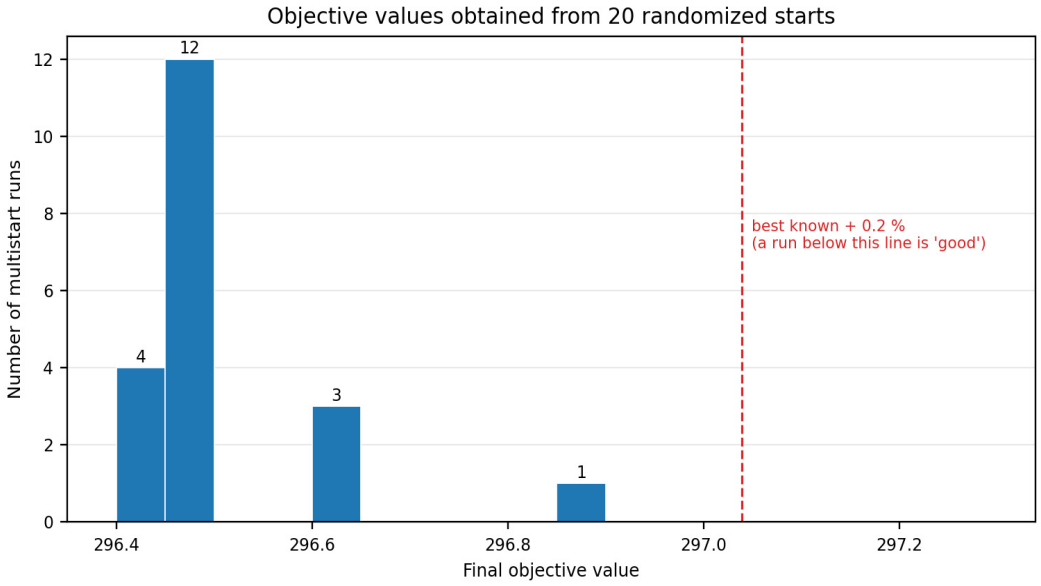


Figure 5. Distribution of final objective values from the 20 randomized starts on af-1.

The distribution is concentrated at 296.45, where 16 of the 20 runs ended; the best run reached 296.44604 and the worst 296.893 (0.15% higher), so all runs lie inside the 0.2% ‘good’ band. This is an important validation result because the purpose of randomized multistart is to explore different local basins and retain the best feasible solution found within the computational budget.

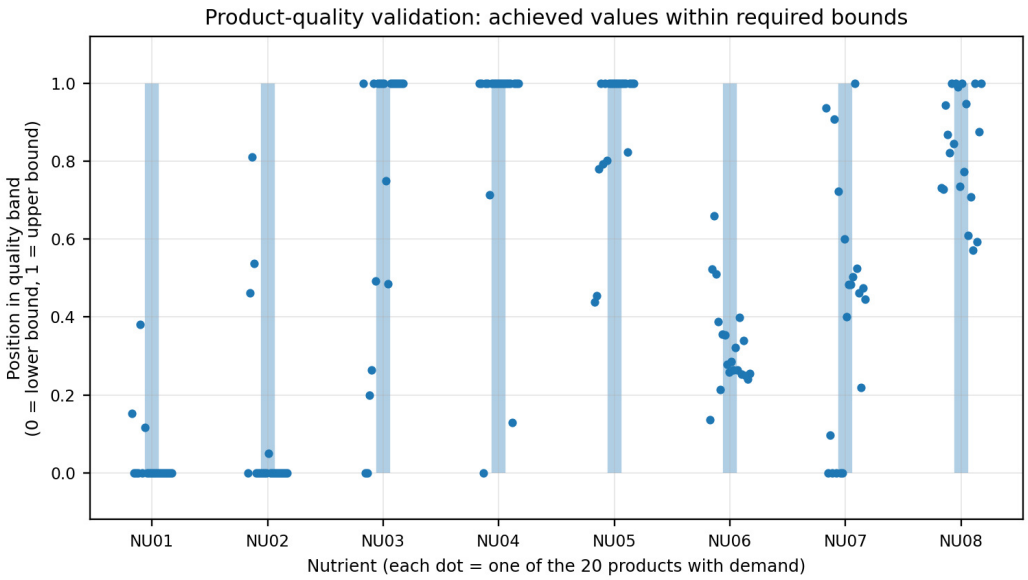


Figure 6. Achieved nutrient levels of the 20 products with demand (best run, af-1), as position within their quality bounds.

The best solution places all 160 product–nutrient compositions (20 products with demand × 8 nutrients) inside their specified bounds, and 91 of them lie exactly on a bound. The largest violation of any quality bound is about 1.8×10^{-15}, effectively zero at floating-point precision, and the largest residual of the mixing equations is 4.95×10^{-10} in the original units. Therefore, the best solution is feasible for af-1.

Additional numerical evidence: In the best solution all 6 bins are used and 99.9% of the total demand of 400 t is supplied through the bins, the rest as straights. The 20 runs ended in six distinct objective values between 296.44604 and 296.893. The best objective, 296.44604, equals the best known value in Table 1 of the paper, and its gap to the paper's best lower bound (295.93153) is 0.17%. The mixing and demand equations hold up to numerical tolerance.

8. Discussion

The implementation demonstrates the central idea of SLP clearly. The original pooling model is nonlinear because the cost and quality expressions contain products between proportions and flows. Instead of solving that nonlinear model directly in one step, SLP constructs a local linear approximation. The LP subproblem is comparatively simple and can be controlled with a trust region. The original nonlinear model is then used to judge the trial point.

The convergence graph shows that the objective can improve substantially within a small number of iterations. The multistart experiment is equally important: four starts found 296.44604, the paper's best known value, while most stopped at 296.4514. This does not prove global optimality (the best lower bound is 295.93153); it is only the smallest objective among the 20 feasible runs, and SLP is a local method.

The result also illustrates why the selected paper evaluates optimization methods using solution quality and computational time rather than only formal optimality. For practical applications, a high-quality feasible solution obtained quickly can be useful even when a global optimality certificate is unavailable. At the same time, global methods such as spatial branch-and-bound remain important when proof of global optimality is required.

There are clear implementation differences. The published study uses CPLEX and 500 starting points per instance. The present implementation uses analytic Jacobians of the bilinear qq-constraints, SciPy's HiGHS interface for the LP subproblems, attribute scaling, a block restoration step and 20 starting points. The best objective is therefore directly comparable with the paper (296.44604 on af-1 in both), whereas iteration counts are not (72.5 per run here, 38.0 in the paper) and run times depend on the machine and the LP solver.

8.1 Practical Observations

- Feasibility should be checked on the original nonlinear constraints, not only on the linearized subproblem.
- The trust-region radius is an important numerical control because the linear model is only locally accurate.
- Multistart can improve the chance of finding a better local solution in a non-convex problem, but it does not by itself prove global optimality.
- Only af-1 was run here; the complete af-series study (500 starts per instance) is computationally expensive, so the af-1 results are not extrapolated to the other instances.

9. Conclusion

This IA-1 study reviewed six research papers covering matheuristics, sequential linear programming, deterministic global optimization, projected-search interior-point methods, accelerated first-order methods, and stationarity-based active-set optimization. The literature shows that nonlinear constrained optimization can be approached from several directions depending on the required balance among solution quality, computational effort, scalability, robustness, and optimality guarantees.

The pooling optimization study by Grothey and McKinnon was selected for detailed implementation because its problem structure is mathematically clear and its SLP procedure can be demonstrated directly. The fixed-demand pooling formulation contains proportional variables, flows, demand constraints, pool convexity constraints, and nonlinear nutrient-quality relationships. The bilinear structure makes the problem non-convex and allows multiple local solutions.

A Python implementation was developed using NumPy, SciPy, pandas, and Matplotlib, with HiGHS as the LP solver. The algorithm linearizes the bilinear qq-formulation, solves an LP trust-region subproblem, accepts or rejects the trial point with a filter, and restores feasibility when needed. Twenty randomized starts were run on af-1; all were feasible and within 0.2% of the best known value, and the best run obtained 296.44604 (violation about 5×10^{-10}). The product-quality visualization confirms that the achieved compositions satisfy the bounds.

The results reproduce the best known objective of the published af-1 benchmark but are not an exact reproduction of the published tables: the published work uses CPLEX and 500 starts per instance, whereas this IA-1 uses HiGHS and 20 starts on af-1 only, because the complete study is computationally expensive, so run times, iteration counts and %Good differ. The implementation provides evidence of algorithm understanding and execution while keeping a clear boundary between published results and the student experiment.

Overall, the study demonstrates how a difficult nonlinear optimization problem can be approached iteratively through local linear models, trust-region control, and randomized starting points. It also reinforces an important principle from the wider literature: optimization methods should be evaluated in relation to the structure and practical requirements of the problem rather than by algorithm name alone.

References

[1] M. Terzi, Y. Ouazene, A. Yalaoui, and F. Yalaoui, "Matheuristics vs. metaheuristics for joint lot-sizing and dynamic pricing problem with nonlinear demands," *Computers & Operations Research*, vol. 163, 106507, 2024. doi:10.1016/j.cor.2023.106507.

[2] A. Grothey and K. McKinnon, "On the effectiveness of sequential linear programming for the pooling problem," *Annals of Operations Research*, vol. 322, pp. 691–711, 2023. doi:10.1007/s10479-022-05156-7.

[3] S. Sass, A. Mitsos, D. Bongartz, I. H. Bell, N. I. Nikolov, and A. Tsoukalas, "A branch-and-bound algorithm with growing datasets for large-scale parameter estimation," *European Journal of Operational Research*, vol. 316, pp. 36–45, 2024. doi:10.1016/j.ejor.2024.02.020.

[4] P. E. Gill and M. Zhang, "A projected-search interior-point method for nonlinearly constrained optimization," *Computational Optimization and Applications*, vol. 88, pp. 37–70, 2024. doi:10.1007/s10589-023-00549-1.

[5] M. Muehlebach and M. I. Jordan, "Accelerated first-order optimization under nonlinear constraints," *Mathematical Programming*, vol. 215, pp. 407–452, 2026 issue. doi:10.1007/s10107-025-02224-1.

[6] D. di Serafino, W. W. Hager, G. Toraldo, and M. Viola, "On the stationarity for nonlinear optimization problems with polyhedral constraints," *Mathematical Programming*, vol. 205, pp. 107–134, 2024. doi:10.1007/s10107-023-01979-9.

Supporting Implementation Files

Python sources: qq_model.py, slp.py, run_experiments.py, make_report_data.py, make_flowchart.py
Raw multistart results: results/runs_<instance>.csv (one row per run); results/report_af-1/runs.csv
Figures (written by make_report_data.py): slp_flowchart.png, slp_convergence.png, slp_multistart_objectives.png, slp_quality_validation.png

IA-1 Submission Checklist

Checklist Item	Status / File Evidence
Three-member group information is complete.	✓ Group ID 9; Razzan Tambe (15), Saurabh Tripathi (04), Vivek Sharma (12)
Six relevant research papers are included and connected by a common optimization theme.	✓ Section 2 and Table 1
Literature review contains critical comparison, not only paper summaries.	✓ Sections 2.1 and 2.2
Optimization problem is mathematically formulated.	✓ Section 3: variables, objective, constraints, non-convex structure
One research paper is selected and the selection is justified.	✓ Section 4 — Grothey & McKinnon (2023)
Selected methodology and algorithm are explained clearly.	✓ Sections 4.2 and 5; Figure 1
Implementation details and evidence are included.	✓ Section 6; code excerpts and supporting files
Results are reported and compared with the selected study where possible.	✓ Sections 7.1 and 7.2
Discussion and conclusion are supported by the obtained results.	✓ Section 8-9
No original research gap or proposed contribution is claimed in IA-1.	✓ This report re-implements and validates an existing published method only
References are complete and consistently formatted.	✓ References [1]–[6]
Source code and supporting implementation files are placed in the prescribed Google Drive folder.	✓ To be uploaded (report PDF, qq_model.py, slp.py, run_experiments.py, make_report_data.py, make_flowchart.py, results CSVs)